VyroEngine

Architecture, Design, and Engineering Methodology of a Modern Cross-Platform Game Engine Built From Scratch

---

A Technical Research Paper

Gaurav
Principal Engine Architect, VyroEcosystem
gauravganesh1214@gmail.com

Version 1.0 | June 2026

Part of the VyroEcosystem initiative — github.com/Gaurav06120714/VyroEngine

Abstract

Modern interactive entertainment and real-time simulation demand game engines that are simultaneously high-performance, portable, extensible, and approachable. Commercial engines such as Unity and Unreal, and the open-source Godot, each occupy distinct points in this design space. This paper presents **VyroEngine**, a game engine built from first principles in modern C++23, with a Vulkan-first rendering backend, a data-oriented Entity-Component-System (ECS) core, and an integrated tooling pipeline. We articulate the engine's vision of *transparency over magic*, in which internal subsystems are explicit, inspectable, and replaceable rather than hidden behind opaque abstractions.

We describe a twelve-phase development methodology spanning core foundation, ECS architecture, rendering, physics, audio, animation, scripting, editor tooling, networking, advanced graphics, optimization, and production release. For each subsystem we analyze the design constraints, the relevant trade-offs, and how comparable mechanisms are realized in industry-standard engines. We further contribute a complete dependency graph, a solo-developer team structure, a phased 6/12/24-month roadmap, and a quantitative risk analysis. The paper concludes with a reference architecture and a discussion of how a single architect can responsibly scope and sequence the construction of a production-grade engine.

Keywords: game engine, ECS, Vulkan, real-time rendering, physics simulation, scripting, engine architecture, C++23.

Table of Contents

1. Introduction

A game engine is a layered software system that abstracts the hardware and operating-system services required to produce real-time interactive applications. At minimum it must orchestrate a deterministic main loop, a rendering pipeline, an input subsystem, an audio subsystem, and a content pipeline that converts authored assets into runtime-optimized data. Contemporary engines extend this baseline with physics simulation, skeletal animation, scripting virtual machines, networking, and full visual editors.

Building such a system from scratch is an ambitious undertaking traditionally reserved for well-resourced studios. However, three trends have lowered the barrier: (1) the maturation of cross-platform libraries for windowing, audio, and asset import; (2) the standardization of explicit graphics APIs such as Vulkan that expose GPU behavior directly; and (3) the consolidation of data-oriented design patterns — chiefly the Entity-Component-System — that decouple data layout from behavior and enable cache-friendly, parallelizable execution.

VyroEngine is conceived as a **pedagogically transparent yet production-capable** engine. Its guiding philosophy, *transparency over magic*, holds that every subsystem should be explicit, documented, and replaceable. Where commercial engines optimize for designer productivity by hiding complexity, VyroEngine deliberately surfaces it, targeting engine programmers and technically inclined teams who require deterministic, inspectable behavior.

1.1 Contributions

- A complete software requirements specification and layered architecture for a modern engine built in C++23 with a Vulkan-first backend.
- A twelve-phase development methodology decomposed into auditable sub-phases, each with deliverables, technical challenges, and success criteria.
- A comparative analysis situating each subsystem against Unity, Unreal Engine, and Godot.
- A complete dependency graph, solo-developer team structure, and phased 6/12/24-month roadmap.
- A quantitative risk register and a reference architecture diagram.

1.2 Paper Organization

Section 2 surveys the industry landscape. Section 3 formalizes the vision and requirements. Section 4 presents the architecture. Sections 5–10 detail subsystems. Section 11 describes the development methodology and roadmap, Section 12 the risk analysis, and Section 13 concludes.

2. Related Work and Industry Landscape

Three engines anchor the modern landscape and serve as recurring reference points throughout this paper.

2.1 Unity

Unity popularized component-based authoring with a managed C# scripting layer atop a native C++ core. Its strengths are an approachable editor, a vast asset store, and broad platform reach. Historically Unity coupled behavior to GameObjects via the MonoBehaviour pattern; its newer Data-Oriented Technology Stack (DOTS) introduces a true archetype-based ECS and the Burst compiler for SIMD-optimized jobs. Unity demonstrates the value of an accessible editor and a managed scripting experience, but also the costs of a managed runtime and historically opaque internals.

2.2 Unreal Engine

Unreal targets the high-fidelity end of the spectrum. It pairs a C++ gameplay framework with the Blueprint visual scripting system and ships a state-of-the-art deferred renderer featuring Lumen global illumination and Nanite virtualized geometry. Unreal exemplifies a vertically integrated, feature-complete engine, at the cost of a large codebase and a steep learning curve. Its actor/component model and reflection system (UObject) inform VyroEngine's approach to editor-exposed properties.

2.3 Godot

Godot is a fully open-source engine built around a scene-tree of nodes rather than a pure ECS. It is notable for a lightweight footprint, a permissive MIT license, an integrated editor written in the engine itself, and the GDScript language. Godot proves that a small, transparent, community-driven engine can be production-viable. VyroEngine adopts Godot's openness ethos and its convention of writing engine tooling against the engine's own public API.

2.4 Positioning

VyroEngine aims to combine Godot's transparency, Unreal's rendering ambition, and Unity's approachability, while making the data-oriented ECS — rather than a scene-tree or GameObject hierarchy — the structural core. The following table summarizes the comparison.

Dimension	Unity	Unreal	Godot	VyroEngine
Core model	GameObject / DOTS	Actor + Component	Scene tree	Pure ECS
Language	C# + C++	C++ + Blueprint	C++ + GDScript	C++23 + Lua
Renderer	URP/HDRP	Deferred/Lumen	Vulkan/GL	Vulkan-first
License	Proprietary	Proprietary	MIT	MIT
Philosophy	Productivity	Fidelity	Openness	Transparency

Table 1. Comparative positioning of VyroEngine against industry engines.

3. Engine Vision and Requirements

3.1 Vision Statement

VyroEngine is an open-architecture, data-driven game engine that grants developers full control over every layer of the stack — from memory allocators to render passes — while remaining approachable enough for a small team to operate. It privileges predictable performance and explicit control over hidden automation.

3.2 Target Users

- **Engine programmers** who require low-level, inspectable subsystems.
- **Indie and small studios** seeking a royalty-free, source-available engine.
- **Researchers and educators** teaching real-time graphics and systems programming.
- **Technical artists** building custom rendering and tooling pipelines.

3.3 Functional Requirements

ID	Requirement	Priority
FR-1	Deterministic fixed/variable-step main loop	MVP
FR-2	Cross-platform windowing and input abstraction	MVP
FR-3	Data-oriented ECS with archetype storage	MVP
FR-4	2D and 3D rendering via Vulkan with GL fallback	MVP
FR-5	Asset import, hot-reload, and runtime packing	MVP
FR-6	Rigid-body physics with collision resolution	MVP
FR-7	Spatial audio playback and mixing	MVP
FR-8	Lua scripting with hot reload	MVP
FR-9	Integrated editor (hierarchy, inspector, assets)	MVP
FR-10	Skeletal animation with blending	Advanced
FR-11	Client-server networking with replication	Advanced
FR-12	PBR, dynamic shadows, GI, post-processing	Advanced

Table 2. Functional requirements with MVP / advanced classification.

3.4 Non-Functional Requirements

- **Performance:** maintain 60 FPS at 1080p with 100k visible entities on mid-range hardware.
- **Portability:** Windows, Linux, and macOS from a single CMake build.
- **Determinism:** reproducible simulation given identical inputs and seeds.
- **Memory safety:** custom allocators with leak tracking in debug builds.
- **Extensibility:** subsystems behind interfaces, swappable at compile or runtime.

3.5 Platform and Toolchain

Concern	Choice
Language	C++23 (concepts, ranges, modules where supported)
Build system	CMake 3.28+ with Ninja generator
Primary GPU API	Vulkan 1.3

Concern	Choice
Fallback GPU API	OpenGL 4.6 (Metal via MoltenVK on macOS)
Scripting VM	Lua 5.4 / LuaJIT
Windowing	SDL3 / GLFW
Editor UI	Dear ImGui
Asset import	Assimp + custom binary pipeline
Audio	OpenAL Soft / miniaudio

Table 3. Selected platform and toolchain decisions.

3.6 Hardware Requirements

Development targets a workstation with an 8-core CPU, 32 GB RAM, and a Vulkan-1.3-capable GPU. The minimum runtime target is a 4-core CPU, 8 GB RAM, and a GPU exposing Vulkan 1.1 or OpenGL 4.5.

4. System Architecture Overview

VyroEngine adopts a strictly layered architecture. Lower layers have no knowledge of higher layers, and cross-cutting services (logging, memory, events) are injected rather than referenced globally. The layering is summarized below.

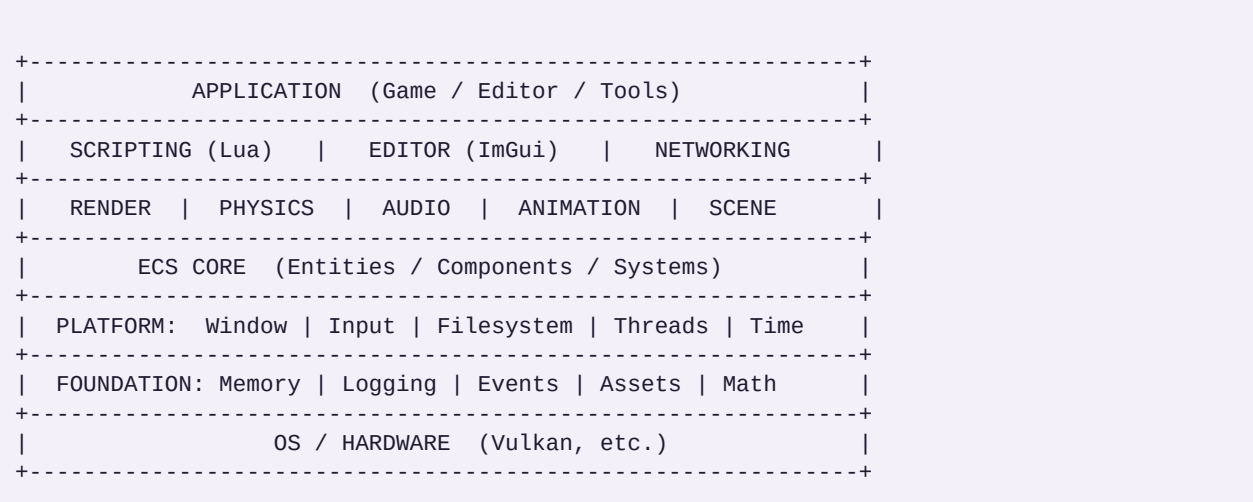

Figure 1. VyroEngine layered architecture. Foundation services are available to all higher layers; the ECS core mediates all gameplay subsystems.

The **Foundation** layer provides allocators, logging, an event bus, asset management, and a math library. The **Platform** layer abstracts OS services. The **ECS core** is the structural heart, storing all simulation state. Gameplay subsystems (render, physics, audio, animation) operate as systems over component data. Higher layers — scripting, editor, networking — compose these primitives.

Data flows through a single frame tick: input is polled, scripts and gameplay systems mutate components during the variable-step update, the physics system advances on a fixed timestep accumulator, and the render system extracts a view of renderable components to build command buffers. This separation of *simulation* from *presentation* is a recurring theme.

5. Core Foundation Subsystems

Phase 1 establishes the foundation upon which all later work depends. Its sub-systems are intentionally small, heavily tested, and free of external dependencies beyond the standard library and chosen platform shims.

5.1 Memory Management

Rather than rely solely on the general-purpose allocator, VyroEngine provides specialized allocators: a linear (arena) allocator for per-frame scratch data, a stack allocator for nested scopes, a pool allocator for fixed-size objects such as components, and a free-list allocator for general allocation. Debug builds wrap these with leak tracking and guard bytes. This mirrors the custom allocator strategies used across AAA engines, where control over memory layout is essential to cache performance.

```
Pool<T> alloc -> O(1) | Arena reset -> O(1) per frame
```

5.2 Logging System

A leveled, category-tagged, thread-safe logger writes to console and rotating files. Compile-time log-level stripping ensures verbose logging incurs zero cost in shipping builds. Structured fields enable later ingestion by tooling.

5.3 Event System

A decoupled publish-subscribe event bus allows subsystems to communicate without direct references. Events are dispatched either immediately or queued for end-of-frame processing. This underpins input propagation, window events, and gameplay messaging, echoing Unreal's delegate system and Godot's signals.

5.4 Input System

Input is abstracted into devices (keyboard, mouse, gamepad) and an action-mapping layer that binds physical inputs to logical actions. This indirection enables rebinding and platform portability, comparable to Unity's Input System package and Unreal's Enhanced Input.

5.5 Asset Management

The asset manager provides reference-counted handles, asynchronous loading, and a virtual filesystem that mounts both loose files (development) and packed archives (shipping). Importers convert source formats into engine-native binary representations. A content hash enables hot-reload: when a source file changes, dependents are notified via the event bus.

Subsystem	Key Data Structure	Primary Concern
Memory	Arena / Pool / Free-list	Cache locality, no leaks
Logging	Lock-free ring buffer	Zero-cost in release
Events	Typed dispatch table	Decoupling
Input	Action map	Rebinding, portability
Assets	Handle + ref-count	Async load, hot-reload

Table 4. Foundation subsystem summary.

6. Entity-Component-System Design

The ECS is VyroEngine's defining architectural choice. Entities are lightweight identifiers; components are plain-old-data structs; systems are functions that iterate over entities possessing a given component signature. This separation yields three benefits: cache-friendly contiguous storage, trivial parallelization of independent systems, and composition over inheritance.

6.1 Entity Management

Entities are 64-bit handles encoding an index and a generation counter. The generation invalidates stale handles when an index is recycled, preventing use-after-free at the gameplay level. A free-list recycles destroyed entity indices.

```
Entity = { uint32 index; uint32 generation; }
```

6.2 Component Storage

VyroEngine uses **archetype** storage: entities sharing the same component signature are grouped into chunks of tightly packed component arrays. Iterating a system therefore walks contiguous memory, maximizing cache-line utilization. Adding or removing a component migrates the entity between archetypes. This is the same strategy adopted by Unity DOTS and the EnTT library frequently used in open-source engines.

6.3 System Manager

Systems declare read and write component dependencies. The scheduler builds a dependency graph and dispatches non-conflicting systems across worker threads, while serializing systems that write shared component types. Fixed-step systems (physics) run on an accumulator independent of variable-step systems (input, animation sampling).

6.4 Scene Management

A scene is a serializable collection of entities and their components. Scenes support additive loading, streaming, and prefab instantiation. Serialization uses a reflective schema so that the editor can save and load arbitrary component types without bespoke code per type.

Aspect	VyroEngine ECS	Unity DOTS	Godot (Nodes)
Storage	Archetype chunks	Archetype chunks	Object tree
Identity	Index+generation	Entity struct	Node pointer
Iteration	Linear over chunks	Linear (Burst)	Tree traversal
Parallelism	Dependency graph	Job system	Limited

Table 5. ECS storage strategies compared.

7. Rendering Engine

Rendering is the most complex subsystem and is built atop a thin Render Hardware Interface (RHI) that abstracts the underlying GPU API. The RHI exposes buffers, textures, pipelines, and command queues, with concrete backends for Vulkan (primary) and OpenGL (fallback).

7.1 Render Hardware Interface

By isolating API specifics behind the RHI, the renderer and all higher layers remain API-agnostic. Vulkan's explicit model — manual synchronization, descriptor sets, and command-buffer recording — is encapsulated so that gameplay code never touches a VkDevice. This mirrors Unreal's RHI and Godot's RenderingDevice abstraction.

7.2 Renderer Architecture

The renderer follows an **extract–prepare–submit** pattern. During *extract*, the render system reads renderable components into an immutable render-view, decoupling it from simulation. During *prepare*, draw calls are sorted and batched by material and depth to minimize state changes. During *submit*, command buffers are recorded — potentially on multiple threads — and presented.

7.3 Shader System

Shaders are authored in GLSL and compiled to SPIR-V offline; a runtime reflection step extracts descriptor layouts so that material parameters bind automatically. A shader-permutation system generates variants for feature toggles (e.g., shadows on/off).

7.4 Texture and Material System

Textures support mipmapping, compression (BCn), and streaming. Materials bind a shader to a set of textures and uniform parameters, exposed to the editor through reflection.

7.5 Camera System

Cameras provide perspective and orthographic projections, frustum culling, and multiple-viewport support for the editor. View and projection matrices are computed once per frame and uploaded as a uniform buffer.

7.6 2D and 3D Renderers

The 2D renderer batches sprites into dynamic vertex buffers with texture atlasing for high throughput. The 3D renderer performs mesh rendering with frustum culling and, in advanced phases, a deferred path supporting physically based shading. Both share the RHI and material systems.

```
Extract  ->  Cull  ->  Sort/Batch  ->  Record CmdBuf  ->  Submit  ->  Present
```

Figure 2. The render frame pipeline.

Stage	Responsibility	Threading
Extract	Snapshot renderable state	Main thread
Cull	Frustum / occlusion rejection	Job system
Sort/Batch	Minimize state changes	Job system
Record	Build command buffers	Multi-thread
Submit	Queue submit + present	Render thread

Table 6. Render pipeline stages and their threading model.

8. Physics, Audio, and Animation

8.1 Physics Engine

The physics subsystem advances on a fixed timestep to ensure determinism. Collision detection proceeds in two phases: a **broad phase** using a dynamic bounding-volume hierarchy or spatial hash to cull non-colliding pairs, and a **narrow phase** using the GJK/EPA algorithms for convex shapes to produce contact manifolds. Collision resolution applies a sequential-impulse solver that iteratively corrects velocities and positions, with constraints (joints) layered atop the same solver.

Rigid bodies carry mass, inertia tensors, restitution, and friction. A debug-draw layer visualizes colliders, contact points, and constraint anchors. While VyroEngine implements a custom solver for transparency, the architecture also permits integrating Bullet3 as a drop-in backend, comparable to how Unity wraps PhysX and Godot offers both a built-in and a Jolt backend.

Phase	Technique	Output
Broad phase	Dynamic BVH / spatial hash	Candidate pairs
Narrow phase	GJK + EPA	Contact manifold
Resolution	Sequential impulse solver	Corrected velocities
Constraints	Joints on impulse solver	Stable articulation

Table 7. Physics pipeline stages.

8.2 Audio Engine

The audio engine streams and decodes sources, mixes them through a bus graph with per-bus effects, and applies spatialization. Spatial audio attenuates and pans sources according to listener position, with optional HRTF for headphone immersion. A resource layer manages decoded buffers and streaming sources, paralleling the foundation asset manager.

- **Playback:** streamed and one-shot sources with sample-rate conversion.
- **Mixing:** hierarchical bus graph with volume, EQ, and reverb sends.
- **Spatialization:** distance attenuation, panning, Doppler, optional HRTF.
- **Resources:** ref-counted decoded buffers and streaming handles.

8.3 Animation System

Skeletal animation samples keyframed bone transforms, builds a pose, and uploads a skinning-matrix palette consumed by vertex shaders. An animation **blend tree** mixes multiple clips (e.g., walk/run by speed), and a **state machine** governs transitions between animation states driven by gameplay parameters. This design follows the Mecanim model in Unity and the AnimGraph in Unreal.

```
Pose = sample(clip, t)  ->  blend(poses, weights)  ->  skinningMatrices[]
```

Figure 3. Animation evaluation pipeline from clip sampling to skinning palette.

9. Scripting and Editor Tooling

9.1 Scripting System

Gameplay logic is exposed to Lua, chosen for its small footprint, embeddability, and performance (especially under LuaJIT). The engine binds a curated Script API surface — entity creation, component access, input queries, and event subscription — rather than exposing raw engine internals. Scripts hot-reload on file change, preserving iteration speed. The boundary between native and scripted code is explicit, avoiding the performance cliffs of fine-grained interop.

- **Lua integration:** a sandboxed VM per world with controlled global state.
- **Script API:** stable, versioned bindings generated from reflection metadata.
- **Hot reload:** file-watch triggers re-execution while preserving entity state.

This positions Lua analogously to GDScript in Godot and C# in Unity: a productive iteration language layered over a native core. Performance-critical paths remain in C++.

9.2 Editor Development

The VyroEditor is built with Dear ImGui and runs against the engine's public API — embodying the same dogfooding principle that makes Godot's editor an engine application. Its panels are composable docking windows.

- **Scene hierarchy:** a tree view of entities with drag-and-drop reparenting.
- **Inspector:** reflection-driven editing of component fields, with undo/redo.
- **Asset browser:** a filesystem view with import settings and previews.
- **Gizmos:** translate/rotate/scale manipulators rendered in the viewport.
- **Viewport:** an interactive scene view with editor camera controls.

The reflection system is the linchpin connecting ECS components, serialization, scripting bindings, and the inspector: a single schema description drives all four, avoiding duplicated boilerplate. This is analogous to Unreal's UObject reflection (UPROPERTY) and Unity's serialized fields.

Editor Panel	Backed By	Industry Analogue
Hierarchy	ECS entity graph	Unity Hierarchy
Inspector	Reflection schema	Unreal Details panel
Asset browser	Virtual filesystem	Unity Project window
Gizmos	Editor render pass	Unreal viewport tools

Table 8. Editor panels and their backing systems.

10. Networking and Advanced Graphics

10.1 Networking

VyroEngine targets an authoritative client-server model. The server simulates the canonical world state; clients send inputs and render replicated state. **Replication** serializes a delta of relevant component state to each client based on relevance filtering (e.g., spatial interest management). **Synchronization** techniques — client-side prediction, server reconciliation, and entity interpolation — mask latency. This mirrors the replication graph in Unreal and Unity's Netcode for Entities.

Concern	Technique
Topology	Authoritative client-server
State transfer	Delta-compressed component replication
Relevance	Spatial interest management
Latency hiding	Prediction + reconciliation + interpolation

Table 9. Networking design choices.

10.2 Advanced Graphics

The advanced graphics phase elevates fidelity once the Vulkan backend is hardened.

- **PBR:** a metallic-roughness physically based shading model with image-based lighting.
- **Shadows:** cascaded shadow maps for directional lights and cube maps for point lights.
- **Post-processing:** a configurable stack — tone mapping, bloom, FXAA/TAA, color grading.
- **Global illumination:** an approximate dynamic GI solution (e.g., voxel- or probe-based) inspired by Unreal's Lumen.

These features are deliberately deferred until the core renderer, ECS, and tooling are stable, reflecting the principle that fidelity should not precede foundational correctness.

10.3 Optimization

A dedicated optimization phase introduces an instrumented profiler (CPU/GPU timings, frame graph), memory-usage analysis, and rendering optimizations such as GPU-driven culling and draw-call batching. Optimization is data-driven: changes are justified by profiler measurements, never speculation.

11. Development Methodology and Roadmap

Construction proceeds through thirteen phases (0–12), each gated by explicit success criteria. The phases follow a strict dependency order: foundation precedes ECS, ECS precedes rendering and physics, and tooling follows a functional runtime.

Phase	Focus	Est. Duration
0	Research & planning	2–3 weeks
1	Core foundation	6–8 weeks
2	ECS architecture	4–6 weeks
3	Rendering engine	8–12 weeks
4	Physics engine	6–8 weeks
5	Audio engine	3–4 weeks
6	Animation system	4–6 weeks
7	Scripting system	3–4 weeks
8	Editor development	8–10 weeks
9	Networking	6–8 weeks
10	Advanced graphics	10–14 weeks
11	Optimization	4–6 weeks
12	Production release	4–6 weeks

Table 10. Phase breakdown with indicative solo-developer durations.

11.1 Phased Timelines

6-month horizon: Phases 0–3 — a foundation, ECS, and a working 2D/3D renderer capable of displaying a scene. **12-month horizon:** Phases 4–8 — physics, audio, animation, scripting, and a usable editor, yielding a minimum-viable engine. **24-month horizon:** Phases 9–12 — networking, advanced graphics, optimization, and a documented, packaged production release.

11.2 Solo-Developer Team Structure

Operated by a single architect wearing rotating hats: *systems engineer* (foundation, ECS), *graphics engineer* (RHI, renderer), *tools engineer* (editor), and *release engineer* (CI, packaging). Work is sequenced so only one role is active at a time, with each phase committed and tagged independently per the project's Git rules.

11.3 Engineering Standards

The repository codifies engineering rules (the *rulz*): coding standards, ECS conventions, rendering guidelines, memory discipline, performance budgets, testing requirements, Git workflow, security, and commit-authorship policy. These rules, modeled on the contribution standards of mature open-source engines, ensure consistency as the codebase grows.

12. Risk Analysis

The dominant risks are scope, Vulkan complexity, and solo-developer bandwidth. Each is rated by likelihood and impact, with a mitigation.

Risk	Likelihood	Impact	Mitigation
Scope creep	High	High	Strict phase gates; MVP-first cut line
Vulkan complexity	High	High	OpenGL fallback first; thin RHI
Solo bandwidth	Medium	High	Time-boxed phases; reuse libraries
Performance misses	Medium	Medium	Profiler-driven optimization phase
Editor over-engineering	Medium	Medium	ImGui; defer polish
Asset pipeline churn	Medium	Medium	Stable binary format early
Burnout	Medium	High	Sustainable cadence; visible milestones
Dependency rot	Low	Medium	Pin versions; vendored deps

Table 11. Risk register with mitigations.

The single most important mitigation is the **MVP cut line**: Phases 0–8 constitute a self-contained, shippable engine. Phases 9–12 are explicitly deferred so that an incomplete advanced feature never blocks a usable release. This staging converts an open-ended research project into a sequence of bounded, demonstrable deliverables.

12.1 Dependency Graph (Critical Path)

```
P0 -> P1 -> P2 -> P3 -> {P4, P5, P6, P7} -> P8 -> P9 -> P10 -> P11 -> P12
```

Figure 4. Critical-path dependency ordering. Phases 4–7 may interleave once the ECS (P2) and renderer (P3) are stable.

13. Conclusion and Future Work

This paper presented VyroEngine, a modern game engine designed and sequenced from first principles. We argued that a data-oriented ECS core, a Vulkan-first RHI, and a reflection system unifying serialization, scripting, and the editor together form a coherent foundation that balances performance with approachability. We grounded each design decision against the practices of Unity, Unreal, and Godot, and we contributed a complete methodology, dependency graph, roadmap, and risk analysis suitable for a solo architect.

VyroEngine's central thesis is that *transparency is a feature*: by making subsystems explicit and replaceable, the engine becomes both a production tool and a learning artifact. The phased methodology ensures that, at every milestone, a runnable engine exists — never a perpetually half-built monolith.

13.1 Future Work

- GPU-driven rendering with mesh shaders and bindless resources.
- A job-graph scheduler with work-stealing for finer-grained parallelism.
- Virtualized geometry and a global-illumination solution at Lumen/Nanite fidelity.
- A visual scripting layer atop the Lua bindings for non-programmers.
- Console and WebAssembly platform backends.

By following the roadmap herein, a single dedicated architect can progress from an empty repository to a documented, packaged engine release within a 24-month horizon, with a usable MVP available at the 12-month mark.

Appendix A. Repository and Folder Architecture

The repository is organized to separate engine code, tooling, samples, and documentation, enabling independent compilation units and clear ownership boundaries.

```
VyroEngine/
+-- engine/
|   +-- foundation/     (memory, log, events, assets, math)
|   +-- platform/       (window, input, fs, threads, time)
|   +-- ecs/            (entity, component, system, scene)
|   +-- render/         (rhi, renderer, shader, material)
|   +-- physics/        (broadphase, narrowphase, solver)
|   +-- audio/          (playback, mixer, spatial)
|   +-- animation/      (skeleton, blend, statemachine)
|   +-- scripting/      (lua vm, bindings, hotreload)
|   +-- net/            (transport, replication, sync)
+-- editor/             (imgui panels, gizmos, viewport)
+-- runtime/            (game executable entry point)
+-- tools/              (asset pipeline, shader compiler)
+-- samples/            (demo scenes and mini-games)
+-- tests/              (unit + integration tests)
+-- docs/               (papers, design docs, diagrams)
+-- rulz/               (engineering standards)
+-- CMakeLists.txt
+-- README.md
```

Figure 5. VyroEngine repository and folder architecture.

A.1 Git Workflow

The project follows trunk-based development on *main* with short-lived feature branches. Each completed sub-phase is committed atomically and tagged (e.g., *v0.3.0-renderer*). Commits carry no AI or tool co-authorship per the repository's commit-authorship rule, and follow the Conventional Commits convention (*feat:*, *fix:*, *docs:*, *chore:*).

Convention	Rule
Branching	Trunk-based; feature branches < 3 days
Commits	Conventional Commits; atomic; signed
Tagging	Semantic version per completed sub-phase
Authorship	Human-only; no AI/bot co-authors
CI	Build + test on every push (Win/Linux/macOS)

Table 12. Git workflow conventions.

Appendix B. Detailed Phase Deliverables

Each phase below lists its primary goal, key deliverables, and the success criterion that gates progression to the next phase.

Phase	Key Deliverables	Success Criterion
0 Planning	SRS, architecture, roadmap, rulz	Approved design docs
1 Foundation	Allocators, logger, events, input, assets	Window opens; logs; loads asset
2 ECS	Entity mgr, archetype storage, scheduler	100k entities iterate at 60 FPS
3 Render	RHI, 2D+3D renderer, shaders, camera	Textured 3D scene on screen
4 Physics	Broad/narrow phase, solver, debug draw	Stable stacking of rigid bodies
5 Audio	Playback, mixer, spatialization	3D positional sound plays
6 Animation	Skeletal anim, blend tree, FSM	Character blends walk/run
7 Scripting	Lua VM, API, hot reload	Gameplay scripted in Lua
8 Editor	Hierarchy, inspector, assets, gizmos	Author a scene without code
9 Network	Client-server, replication, sync	Two clients share a world
10 Adv. GFX	PBR, shadows, post-fx, GI	PBR scene with dynamic shadows
11 Optimize	Profiler, mem + render optimization	Meets frame-time budgets
12 Release	Tests, docs, packaging, deploy	Tagged, packaged 1.0 build

Table 13. Per-phase deliverables and gating success criteria.

B.1 Skills Matrix

The phases collectively demand a broad skill set, summarized below for capacity planning.

Domain	Skills Exercised	Heaviest Phases
Systems	C++23, memory, concurrency	1, 2, 11
Graphics	Vulkan, GLSL, linear algebra	3, 10
Simulation	Collision math, numerical solvers	4, 6
Tooling	ImGui, UX, reflection	8
Networking	Sockets, serialization, latency	9
Release	CMake, CI/CD, docs, packaging	12

Table 14. Skills matrix across the development lifecycle.

Taken together, Appendices A and B translate the architectural vision of the preceding sections into an actionable, auditable program of work — the bridge between design intent and day-to-day engineering execution.

References

[1] Gregory, J. *Game Engine Architecture*, 3rd ed. CRC Press, 2018.

[2] Nystrom, R. *Game Programming Patterns*. Genever Benning, 2014.

[3] Akenine-Möller, T., Haines, E., Hoffman, N. *Real-Time Rendering*, 4th ed. CRC Press, 2018.

[4] The Khronos Group. *Vulkan 1.3 Specification*. 2022.

[5] Unity Technologies. *Entities (DOTS) Documentation*. 2023.

[6] Epic Games. *Unreal Engine 5 Documentation: Rendering, Lumen, Nanite*. 2023.

[7] Linietsky, J., Manzur, A., et al. *Godot Engine Documentation*. 2023.

[8] Ericson, C. *Real-Time Collision Detection*. Morgan Kaufmann, 2005.

[9] Catto, E. "Iterative Dynamics with Temporal Coherence." GDC, 2005.

[10] Ierusalimschy, R. *Programming in Lua*, 4th ed. 2016.

[11] Caudron, M., et al. "EnTT: A Fast and Reliable ECS for C++." 2023.

[12] Glassner, A. (ed.) *Graphics Gems*. Academic Press, 1990.

[13] Fiedler, G. "Networked Physics" and "Snapshot Interpolation." Gaffer On Games, 2015.

[14] Pharr, M., Jakob, W., Humphreys, G. *Physically Based Rendering*, 4th ed. 2023.